Java Practical Programs

Department of Computer Science & Engineering — Practical Reference

PLACEHOLDER DOCUMENT NOTICE

This is a verified placeholder document. To replace it with your actual PDF file,
save your original PDF as "public/pdfs/java-practical.pdf" in your project directory.

Table of Practical Programs:

1. Multithreading & Synchronization in Java (Producer-Consumer)
2. TCP/IP Socket Programming (Client-Server Architecture)
3. JDBC Database Connectivity & Transaction Processing
4. Java Collections Framework & Custom Comparators
5. Functional Programming with Java Stream API & Lambdas

Program 1: Multithreading & Synchronization

Objective: Implement thread synchronization using wait() and notify() mechanisms.

```
public class ThreadSyncDemo {
    private static final Object lock = new Object();
    private static int buffer = 0;
    private static boolean hasData = false;

    public static void main(String[] args) throws InterruptedException {
        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                synchronized (lock) {
                    while (hasData) {
                        try { lock.wait(); } catch (InterruptedException e) {}
                    }
                    buffer = i;
                    System.out.println("Produced item: " + buffer);
                    hasData = true;
                    lock.notify();
                }
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                synchronized (lock) {
                    while (!hasData) {
                        try { lock.wait(); } catch (InterruptedException e) {}
                    }
                    System.out.println("Consumed item: " + buffer);
                    hasData = false;
                    lock.notify();
                }
            }
        });

        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}
```

Program 2: TCP/IP Client-Server Socket Communication

Objective: Create a bidirectional communication channel between client and server.

```
import java.io.*;
import java.net.*;

public class EchoServer {
    public static void main(String[] args) throws IOException {
        int port = 8080;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server listening on port " + port);
            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("Client connected: " + socket.getInetAddress());
                BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputS
                PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
                String line;
                while ((line = in.readLine()) != null) {
                    if ("EXIT".equalsIgnoreCase(line.trim())) break;
                    out.println("ECHO: " + line);
                }
                socket.close();
            }
        }
    }
}
```

Program 3: JDBC Database CRUD Operations

Objective: Execute SQL queries using PreparedStatement and handle transactions.

```
import java.sql.*;

public class DatabaseManager {
    private static final String URL = "jdbc:postgresql://localhost:5432/university";
    private static final String USER = "postgres";
    private static final String PASS = "secret";

    public static void insertStudent(int id, String name, double gpa) throws SQLException {
        String sql = "INSERT INTO students (id, name, gpa) VALUES (?, ?, ?)";
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            stmt.setString(2, name);
            stmt.setDouble(3, gpa);
            int rows = stmt.executeUpdate();
            System.out.println("Inserted " + rows + " row(s) successfully.");
        }
    }
}
```

Program 4: Functional Programming with Java Stream API

Objective: Filter, map, and aggregate dataset using Java Stream pipeline.

```
import java.util.*;
import java.util.stream.Collectors;

public class StreamApiDemo {
    record Student(String name, String department, double score) {}

    public static void main(String[] args) {
        List<Student> roster = List.of(
            new Student("Alice", "CS", 92.5),
            new Student("Bob", "IT", 85.0),
            new Student("Charlie", "CS", 88.0),
            new Student("Diana", "ECE", 95.0)
        );

        List<String> topPerformers = roster.stream()
            .filter(s -> s.score() >= 90.0)
            .sorted(Comparator.comparingDouble(Student::score).reversed())
            .map(s -> s.name() + " (" + s.department() + "): " + s.score())
            .collect(Collectors.toList());

        topPerformers.forEach(System.out::println);
    }
}
```